BM GAME API, VOLUME 4

The bmlib and bmnet `libraries`

The shared library for the things every game writes again, and the library for games over the network.

Contents

The two libraries . . . 4

What they are . . . 4

How to load them . . . 4

Good to know about bmlib . . . 4

Good to know about bmnet . . . 4

Tutorial: a platformer step with bmlib . . . 6

Before you start . . . 6

Steps . . . 6

Tutorial: a game for two over the network with bmnet . . . 8

Steps . . . 8

bmlib: numbers . . . 10

bmlib: random numbers . . . 11

bmlib: collisions . . . 12

bmlib: the map, walls, platforms, gravity . . . 13

bmlib: hitboxes and hurtboxes . . . 14

bmlib: easing, tweens, timers and scripts . . . 16

bmlib: lists and particles . . . 17

bmlib: high-throughput 3D and crowds (crowd3d) . . . 18

bmlib: the camera . . . 19

bmlib: one game, many screens . . . 20

bmlib: game states . . . 21

bmlib: text, bars and menus . . . 22

bmlib: players on one console . . . 24

bmlib: sound, saves, animations, colours 25

bmlib: the 3D builder 26

bmnet: opening the network and the lobby 27

bmnet: messages and events 28

bmnet: the match in lockstep 29

bmnet: fields and a whole example 30

Fields 30

A whole example 30

Trying it on the PC 31

Open points 32

README

What this is. Volume 4 of the bm game API. It describes the two standard libraries of the kernel, `bmlib` and `bmnet`: every function, with its signature and what it does.

How to use it. Read the first three chapters once: they load the libraries and build a small platformer and a two-player game over the network, step by step. Then use the other chapters as a reference, one area at a time.

What is missing. Every gap is marked with a highlighted TO FILL tag; nothing is invented.

The two libraries

What they are

`bmlib` is what every game used to write again by itself, now in a single library written in Lua and kept in the kernel (`src/script/bmlib.lua`). It covers: numbers and random numbers, collisions, the map (walls, platforms, gravity), hitboxes and hurtboxes, easing, tweens, timers and scripts, particles, a camera, game states, text, menus and the pause menu, players on one console, jingles, saves, animations, colours and a 3D mesh builder.

`bmnet` is the network code of the game Overbit, made available to every game (`src/script/bmnet.lua`). Consoles find each other on the **LAN** (UDP broadcasts) or over the **internet** through a relay; one console **hosts** a match and the others **join**; they send **messages** and, for action games, the match runs in **lockstep**.

How to load them

```
local lib = require "bmlib"
local net = require "bmnet"
```

`bmlib` does not touch the global variables and hides no function of the console.

Good to know about bmlib

- Times are in **seconds**, positions in pixels, speeds in **pixels per frame**.
- Objects (particles, camera, states, menu, pause, 3D builder) are used with a colon: `cam:follow(x, y)` .
- Tweens, timers, scripts and jingles go on with `lib.update()` , to call once in `_update` . It adds 1/60 s; `lib.update(dt)` adds another step. `lib.time` is the time counted that way.

```
local lib = require "bmlib"

function _update()
  lib.update()          -- tweens, timers, scripts and jingles
  -- ... the game
end
```

Good to know about bmnet

- Call `net.update()` once in each `_update` : it returns the events of the frame.
- Call `net.close()` when the game ends, and from `_leave()` too.

- The packets are described at the top of `src/script/bmnet.lua`.
- The SDK's *Online 2D* template is a whole example of a game over the network. The *Versus 2D* template is a whole example of players on one console.

Tutorial: a platformer step with bmlib

You build a hero that walks, jumps and lands on tiles, with a camera that follows it. You use only functions described in this manual.

Before you start

The map is drawn on layer 1 and its tiles carry **flags**. bmlib reads them like this:

Flag	Meaning	Constant
0	solid	`lib.SOLID`
1	platform	`lib.PLATFORM`
2	ladder	`lib.LADDER`
3	water	`lib.WATER`
4	hurts	`lib.HURT`

[TO FILL: how to paint the tiles and set their flags in the editors; this volume only describes the library]

Steps

1. Start a new Lua file and load the library: `local lib = require "bmlib"`.
2. Make a table for the hero: `local hero = { x = 24, y = 24, w = 8, h = 8 }`. A body is a table with `x, y, w, h`.
3. In `_init`, call `lib.tiles({ edge = true })`. Layer 1, solid flag 0 and platform flag 1 are the defaults; `edge = true` makes the borders of the map walls.
4. In `_init`, make the camera: `cam = lib.camera({ bounds = true })`. With `bounds = true` it stays inside the map.
5. In `_update`, call `lib.update()` first.
6. Set the hero's speed from the pad: `hero.vx` is 2 with `btn("right")`, -2 with `btn("left")`.
7. Jump only when `hero.ground` is true: `hero.vy = -5` when `btnp("a")` is pressed.
8. Set `hero.drop = btn("down")`, so the hero can go down through a platform.
9. Call `lib.step(hero)`. It adds gravity, moves the hero, stops it against the solid tiles and sets `hero.ground`.
10. Call `cam:follow(hero.x, hero.y)`.
11. In `_draw`: `cls(0)`, `cam:apply()`, `cam:map()`, draw the hero, then `camera()` for the fixed text.
12. Run the game. You should see the map, the hero falling onto the ground, and the screen following it when you walk.

```
local lib = require "bmlib"
local hero = { x = 24, y = 24, w = 8, h = 8, vx = 0, vy = 0 }
local cam

function _init()
  lib.tiles({ edge = true })              -- the map's borders are walls
  cam = lib.camera({ bounds = true })     -- stays inside the map
end

function _update()
  lib.update()
  hero.vx = (btn("right") and 2 or 0) - (btn("left") and 2 or 0)
  if hero.ground and btnp("a") then hero.vy = -5 end
  hero.drop = btn("down")                 -- down from a platform
  lib.step(hero)
  cam:follow(hero.x, hero.y)
end

function _draw()
  cls(0)
  cam:apply()
  cam:map()
  print("@", hero.x, hero.y, 0xFFFFFF)    -- the hero
  camera()
  print("A: jump", 4, 4, 0xFFFFFF)
end
```

[TO FILL: shots/api-lib-platformer.png]

TRY IT

Change `hero.gravity` to 0.15, then set `hero.maxfall` to 3, and jump again. The hero falls slower, and never faster than 3 pixels per frame.

If the hero falls through the floor, check that the floor tiles have flag 0 (solid) set, and that they are on layer 1, or pass the right `layer` to `lib.tiles`.

Tutorial: a game for two over the network with bmnet

You build a lobby: one console hosts, the other joins, they talk with messages and then start a match. You use only functions described in this manual. Do it on two consoles on the same LAN, or on the PC (see the last chapter).

Steps

1. Load the libraries: `local net = require "bmnet"` and `local lib = require "bmlib"`.
2. In `_init`, call `net.open({ game = "MYG1", name = "Ann" })`. `game` is 4 characters: the packets of other games are not seen. It returns `true`, or `false` and why. Then `net.state` is `"lobby"`.
3. Add `function _leave() net.close() end`, so the others know when you go.
4. In `_update`, loop over `net.update()`: it returns the events of the frame. Do this once in each `_update`.
5. On the first console, when `net.state` is `"lobby"` and `btnp("a")` is pressed, call `net.host({ max = 2, info = "my match" })`. This console is seat 1 and `net.state` becomes `"hosting"`.
6. On the second console, read `net.hosts()`: it lists the matches found in the last 3 seconds. Show them, and call `net.join(id)` for the one chosen.
7. Wait for the event `"joined"` (it is `"refused"` if the match is full or has begun). The host gets an event `"join"` with the name of the newcomer.
8. Say something: `net.post("hello")` sends a **sure** message. The other console gets an event `"msg"` with `from`, `data` and `sure`.
9. On the host, when `btnp("start")` is pressed, call `net.start()`. Everybody gets the event `"start"` with `seed`, `seat` and `seats`.
10. On `"start"`, make the random generator with `lib.rng(e.seed)`. From now on `net.state` is `"playing"`.
11. In every `_update` of the match: `net.input(net.pad(1))`, then `for f, inputs in net.frames() do ... end`. Move each player from `inputs[seat]`, with `net.held(inputs[seat], "right")`; skip a seat whose input is `false` (the player has left).
12. Run on both consoles. You should see the lobby, the other player's name, the messages, and the two squares moving the same way on both screens.

```
local net = require "bmnet"
local lib = require "bmlib"
local rng
local chat = {}
local pos = { 40, 120 }                       -- one position per seat

function _init() net.open({ game = "MYG1", name = "Ann" }) end
function _leave() net.close() end

local function step(inputs)                   -- the same on every console
  for seat = 1, 2 do
    local v = inputs[seat]
    if v then
      if net.held(v, "right") then pos[seat] = pos[seat] + 1 end
      if net.held(v, "left") then pos[seat] = pos[seat] - 1 end
    end
  end
end

function _update()
  for _, e in ipairs(net.update()) do
    if e.type == "join" then chat[#chat + 1] = e.name .. " joined"
    elseif e.type == "msg" then chat[#chat + 1] = e.data
    elseif e.type == "start" then rng = lib.rng(e.seed)
    elseif e.type == "lost" then chat[#chat + 1] = "host lost" end
  end
  if net.state == "lobby" then
    if btnp("a") then net.host({ max = 2, info = "my match" }) end
    for _, h in ipairs(net.hosts()) do
      if btnp("b") then net.join(h.id) end
    end
  end
  if net.state == "hosting" and btnp("start") then net.start() end
  if net.state ~= "off" and btnp("x") then net.post("hello") end
  if net.state == "playing" then
    net.input(net.pad(1))
    for _, inputs in net.frames() do step(inputs) end
  end
end
```

[TO FILL: shots/api-lib-lobby.png]

BEWARE

In a match the game must depend only on the inputs of the frames and on the seed. Read the rules of lockstep in the chapter "bmnet: the match in lockstep".

TRY IT

Start the game on two consoles. Press A on the first, B on the second, then X on both to see the messages, then Start on the first.

bmlib: numbers

Function	Description
`lib.clamp(v, lo, hi)`	`v` between `lo` and `hi`
`lib.lerp(a, b, t)` / `lib.unlerp(a, b, v)`	from `a` (t = 0) to `b` (t = 1); the reverse: where `v` is between `a` and `b`
`lib.remap(v, a0, a1, b0, b1)`	`v` from the range `a0..a1` to the range `b0..b1`
`lib.approach(v, target, step)`	`v` toward `target` by at most `step` (bars that go down slowly, speeds that brake)
`lib.sign(v)`	-1, 0 or 1 (0 for 0)
`lib.round(v, [step])`	to the nearest integer, or to the nearest multiple of `step`
`lib.wrap(v, lo, hi)`	`v` brought back into `lo..hi` (`hi` excluded) going round: worlds that wrap
`lib.cycle(i, d, n)`	index `i` (1..n) moved by `d` going round: the rows of a menu
`lib.dist(ax, ay, bx, by)` / `lib.dist2(...)`	the distance / its square (to compare it with `r * r` without the root)
`lib.len(x, y)` / `lib.norm(x, y)`	the length of a vector / the vector 1 long (and the length it had); `0, 0, 0` for `0, 0`
`lib.angle(ax, ay, bx, by)`	the angle from `a` to `b` in radians (0 to the right, π/2 down on the screen)
`lib.angdiff(a, b)` / `lib.turn(a, target, step)`	from `a` to `b` the shortest way (-π..π) / angle `a` turned toward `target` by at most `step`
`lib.dir8(x, y)`	one of 8 directions: 0 right, 1 down-right, 2 down... 7 up-right (`nil` for `0, 0`); `lib.DIR8[d + 1]` is the vector 1 long
`lib.TAU`	2π

bmlib: random numbers

Function	Description
`lib.rnd([a, b])`	a number between `a` and `b` (`b` excluded); `lib.rnd(n)`: between 0 and `n`; `lib.rnd()`: between 0 and 1
`lib.chance(p)`	`true` with probability `p`
`lib.choose(t)` / `lib.shuffle(t)`	a random element of the list (`nil` if it is empty) / the list shuffled (in place)
`lib.rng(seed)`	a generator of its own, which always gives the same numbers from the same seed (worlds made from a seed, lockstep network matches): `r:next()` between 0 and 1, `r:range(a, b)`, `r:int(a, b)` (integers, both included), `r:pick(t)`, `r:chance(p)`, `r:seed(s)`

bmlib: collisions

Function	Description
`lib.overlap(ax, ay, aw, ah, bx, by, bw, bh)`	two rectangles (x, y, width, height) touch
`lib.hit(a, b)`	the same for two tables with `x, y, w, h` (`w` and `h` 8 if missing)
`lib.inside(px, py, x, y, w, h)`	the point is in the rectangle
`lib.circles(ax, ay, ar, bx, by, br)` / `lib.circrect(cx, cy, r, x, y, w, h)`	two circles / a circle and a rectangle touch

bmlib: the map, walls, platforms, gravity

The functions look at the **tiles' flags**: `lib.SOLID` (flag 0, value 1), `lib.PLATFORM` (flag 1, 2), `lib.LADDER` (4), `lib.WATER` (8), `lib.HURT` (16). The ladder, water and hurt flags are for your own code: for example, `lib.ray(x0, y0, x1, y1, lib.HURT)` finds the first tile that hurts.

Function	Description
`lib.tiles(opt)`	how the map stops the bodies: `layer` (the layer: number or name, 1), `solid` and `platform` (the flags' masks: 1 and 2; 0 = none), `edge` (`true`: outside the map is solid). Returns the configuration
`lib.solid(x, y, [w, h])`	the rectangle (or the point) touches something solid
`lib.move(b, dx, dy)`	moves body `b = {x, y, w, h}` (`w`, `h` 8 if missing) by `dx, dy`, stopping it against the solid tiles and sliding it along the walls; it falls on a platform only from above (and not with `b.drop`: down through it). In steps shorter than a tile: it never jumps one. Returns `hx, hy`: -1 / 1 where it hit (left / right, up / down), otherwise 0
`lib.step(b)`	one frame of a body with gravity (a platformer): `b.vx, b.vy` in pixels per frame, `b.gravity` (`lib.GRAVITY`, 0.25) and `b.maxfall` (`lib.MAXFALL`, 6); then `b.ground` is `true` on the ground and the speed that hit is 0. Returns what `lib.move` gives
`lib.ray(x0, y0, x1, y1, [mask])`	the first tile with a flag of `mask` (`lib.tiles`' solid one if missing) on the segment: `x, y` where the segment enters it and its cell `mx, my`; `nil` if the way is free (line of sight, bullets). With `lib.tiles{edge=true}` any point outside the map also counts as a hit, whatever the mask

```
local hero = { x = 40, y = 40, w = 8, h = 8, vx = 0, vy = 0 }
lib.tiles({ edge = true })                        -- the map's borders are walls
function _update()
  hero.vx = (btn("right") and 2 or 0) - (btn("left") and 2 or 0)
  if hero.ground and btnp("a") then hero.vy = -5 end
  hero.drop = btn("down")                         -- down from a platform
  lib.step(hero)
end
```

bmlib: hitboxes and hurtboxes

Hurtboxes are where a body can be hit, **hitboxes** where an attack hurts (a punch, a sword, a bullet). A world of hits gathers them in every frame and says who hits whom: teams, one hit per attack, the parts of the body, the lanes of a beat 'em up, blades that clash. The boxes are given in the code or come from the sheet (`zboxes`).

Function	Description
`lib.hits()`	a world of hits `H` (tables `H.hurts` , `H.hitl` : the frame's boxes)
`H:clear()`	a new frame, to call in `_update` before the boxes: those of before go; the attacks (`id`) that did not come back in the last frame are over
`H:hurt(who, x, y, w, h, [opt])`	a hurtbox of `who` (any value: the body's table). `team` (the same team does not hit itself), `part` (a name: `"head"` ; the parts that count most first), `z` and `depth` (a third dimension: the lane, the height)
`H:hit(who, x, y, w, h, [opt])`	a hitbox of the attacker `who` : `team` , `id` (an attack: it hits each body **once** while the same `id` comes back frame after frame; the boxes of one attack share it; without `id` it hurts in every frame it touches), `clash` (two hitboxes with `clash` that touch make a `"clash"` contact), `z` , `depth` and whatever the game needs (`damage` , `knock` ...)
`H:zone(who, name, frame, x, y, [flip, opt])`	the boxes of a zone's frame (`zboxes` : `hurt` and `hit`) for the sprite drawn with `zspr(name, x, y, frame, flip)` : the hurtboxes with `opt` (`team` , `part`), the hitboxes with `opt.attack` (the options of `H:hit` ; the team of `opt` if missing)
`H:check()`	the contacts of the frame, in the order of the hitboxes: `{kind = "hit" or "clash", by (the attacker), to (the body hit, or the other attacker), hit (the options of H:hit), part, x, y (the middle of where they touch)}` . An attack hits a body once a frame (its first box that touches)
`H:draw([hurt_c, hit_c])`	the boxes, to see them while making the game (blue and red)
`lib.box(b, x, y, [flip, w])`	a box of a frame (`b.x` , `b.y` from the corner, `b.w` , `b.h`) in the world, for the frame drawn at `x, y` ; with `flip` mirrored over the frame's width `w` : `x, y, w, h`
`lib.separate(a, b)`	two bodies `{x, y, w, h}` that overlap move apart along the axis where they overlap less, half each (`a.fixed` or `b.fixed` : only the other); `true` if they touched. For fighters that do not pass through each other, crowds

```
local H = lib.hits()

function _update()
  H:clear()
  for _, f in ipairs(fighters) do
    H:zone(f, f.anim, f.frame, f.x, f.y, f.face < 0,
           { team = f.team, attack = { id = f.swing, damage = 5 } })
  end
  for _, c in ipairs(H:check()) do
    c.to.life = c.to.life - c.hit.damage    -- c.by hit c.to
  end
end
```

bmlib: easing, tweens, timers and scripts

`lib.ease` has the curves from 0 to 1: `linear`, `inquad`, `outquad`, `inoutquad`, `incubic`, `outcubic`, `inoutcubic`, `insine`, `outsine`, `inoutsine`, `inback`, `outback`, `inoutback`, `outelastic`, `outbounce`, `smooth` (smoothstep).

Function	Description
`lib.tween(obj, to, secs, [ease, done])`	the fields of `obj` go to the values of table `to` in `secs` seconds (`ease`: a function or the name of one of `lib.ease`, linear if missing), then `done(obj)`. Returns a handle: `h:cancel()`
`lib.after(secs, f)`	`f()` in `secs` seconds
`lib.every(secs, f, [times])`	`f()` every `secs` seconds (`times` times, for ever if missing); `f` returning `false` stops it
`lib.script(f, ...)`	`f(...)` as a **script that can wait**: `lib.wait(secs)` (a frame if missing) and `lib.waitfor(cond)` (until `cond()` is true). It runs at once up to its first wait, then `lib.update()` takes it on: dialogues, cutscenes, waves of enemies written in a row
`lib.cancel(h)`	stops a tween, a timer or a script (the same as `h:cancel()`)
`lib.countdown(t, keys, [d])`	the fields `keys` of `t` above 0 go down by `d` (1/60 if missing), not under 0: a hero's cool-downs (in frames with `d = 1`)
`lib.clear()`	stops every tween, timer, script and jingle (a new level)

All of them go on only with `lib.update()`.

```
lib.tween(title, { y = 80 }, 0.6, "outback")
lib.after(2, function() door.open = true end)
lib.script(function()
  say("Who goes there?")
  lib.wait(1.5)
  lib.waitfor(function() return btnp("a") end)
  say("You may pass.")
end)
```

bmlib: lists and particles

Function	Description
`lib.each(list, f)`	`f(item, i)` for each item; those for which it returns `false` leave the list (the others keep their order)
`lib.sweep(list)`	takes away the items with `.dead` (in order)
`lib.particles([max])`	a pool of at most `max` particles (200); when it is full a new one takes the place of the oldest
`P:add(x, y, vx, vy, life, colour, [opt])`	one particle (speeds in pixels per frame, life in seconds); `opt`: `size` (radius; 0 or 1 a pixel), `gravity`, `drag` (the part of the speed kept every frame), `colors` (a list: the colour through its life), `shrink`, `floor` (the y where it bounces), `bounce` (0.3)
`P:burst(x, y, n, [opt])`	`n` particles from `x, y`: `speed` (2), `angle` and `spread` (in radians; all round if missing), `life` (0.5 s), `color` or `colors`, and the options of `:add`
`P:update()` / `P:draw()`	in `_update` / in `_draw`
`P:count()` / `P:clear()`	how many there are / all away
`lib.particles3d([max])`	the 3D sibling of `lib.particles`: a pool of at most `max` particles (1024). It wraps the 3D particle functions (`particles`, `particles_add`, `particles_step`, `particles_draw`, `particles_count`)
`P3:add(x, y, z, vx, vy, vz, life, color)`	one 3D particle
`P3:burst(x, y, z, n, [opt])`	`n` particles from `x, y, z`: `speed` (2), `life` (30 updates), `color` (0xFFFFFF), `up` (0)
`P3:update([gravity])` / `P3:draw([radius])`	in `_update` (gravity 0.02) / in `_draw` (radius 0.1)
`P3:count()`	how many 3D particles are alive

```
local fx = lib.particles(300)
fx:burst(x, y, 20, { speed = 3, colors = { 0xFFFFFF, 0xFFD050, 0xFF6020 }, gravity = 0.1 })

local p3 = lib.particles3d(500)
p3:burst(x, y, z, 20, { speed = 2, life = 30, color = 0xFFA020 })
p3:update()
p3:draw()
```

bmlib: high-throughput 3D and crowds (crowd3d)

Function	Description
`lib.gpu3d_fast([opt])`	activates the high-throughput GPU 3D pipeline: Vertex Shader 2, Command Queue 2, and frame-later pipelining. Pushes up to 50,000 triangles at 60 fps on the Pi Zero W
`lib.crowd3d(mesh, [opt])`	creates a high-throughput 3D crowd and instance renderer with pose bucketing (skeletal animation deduplication); `opt` : `clip` ("walk"), `poses` (16), `radius` (1.3)
`CR:add(x, y, z, [yaw, scale, phase, flags])`	adds an instance to the crowd and returns its `id` index
`CR:set(id, [x, y, z, yaw])`	updates the instance's coordinates and rotation
`CR:snap_to_map(map)`	batch snaps all `y` elevations to a `map3d` terrain, caching them to avoid map queries during drawing
`CR:clear()`	removes all instances
`CR:draw([time, default_flags])`	performs `visible3d` culling, groups instances by quantized pose, evaluates `animate()` once per bucket, and renders instances

```
local lib = require "bmlib"
lib.gpu3d_fast()

local crowd = lib.crowd3d(model("hero0"), { clip = "walk", poses = 16 })
for i = 1, 50 do
  crowd:add(math.random(10, 22), 0, math.random(10, 22), math.random() * 6.28)
end
crowd:snap_to_map(world_map)

function _draw()
  cls(0x8CC8F0)
  zclear()
  camera3d(cx, cy, cz, cyaw, cpitch, 60)
  map3d_draw(world_map)
  crowd:draw(time())
end
```

bmlib: the camera

Function	Description
`lib.camera([opt])`	a 2D camera: `smooth` (0.15: the part of the way it goes every frame; 1 = it sticks to the target), `dead` `{w, h}` (a box in the middle where the target moves without the camera), `bounds` `{x0, y0, x1, y1}` in pixels or `true` (the map, `msize()`), `offset` `{x, y}`, `w`, `h` (the screen if missing). Fields `x`, `y`
`C:follow(x, y, [snap])`	one step toward point `x, y` (in the middle of the screen); `snap`: there at once
`C:shake(amount, [secs])`	the screen shakes up to `amount` pixels, less and less, for `secs` seconds (0.3)
`C:apply([view])`	`camera()` at the camera's place (with the shake): in `_draw` before the world, then `camera()` for the HUD. With a view `{x, y, w, h}` (`lib.split`) the drawing stays in the view (`clip`) and the camera's corner is the view's: after the views, `clip()` and `camera()`
`C:map([layer, mask])`	the cells of the map that show
`C:sees(x, y, [w, h])` / `C:screen(x, y, [view])`	the rectangle shows / where a point of the world is on the screen (in the view)

bmlib: one game, many screens

Function	Description
`lib.view()`	a table `{x, y, w, h, scale, wide, square, tall}`. `x, y, w, h` are the safe area. `scale` is how many screen pixels one design pixel gets (`1.0` on the design size). `wide`, `square`, `tall`: the shape of the screen, one is `true`
`lib.anchor(where, w, h, [mx, my])`	`x, y` of an element `w` by `h` at `"tl"` `"t"` `"tr"` `"l"` `"c"` `"r"` `"bl"` `"b"` `"br"` of the safe area, with margins `mx, my` (0) from the edges it touches
`lib.safe_clip()`	`clip()` to the safe area; returns `x, y, w, h`
`lib.fit(dw, dh)`	`scale, x, y` to draw a layer made for `dw` by `dh` whole, in the middle of the safe area

Call them in `_draw` or in `_screen(w, h)`, not once at load: `screen()` can change the size.

bmlib: game states

`lib.states(defs, [first, ...])` : title, match, pause, game over as tables `{enter, update, draw, exit}` ; each function gets its table (`function play:update()`), which has `t` (the seconds spent in the state) and `name` .

Function	Description
`S:go(name, ...)`	to state `name` : the open ones leave (`exit`), `name` enters (`enter(...)`)
`S:push(name, ...)` / `S:pop()`	`name` over the one now (a pause, a dialogue: only it updates, both draw) / back to the one under it
`S:update()` / `S:draw()`	in `_update` / in `_draw` (from the bottom: the game under its pause)
`S:is(name)` , `S.name` , `S:top()`	the state on top

```
local S
S = lib.states({
  title = { update = function(s) if btnp("ok") then S:go("play") end end,
            draw = function() cls(0); lib.printc("PRESS A", 160, 0xFFFFFF) end },
  play = { enter = function(s) s.score = 0 end,
           update = function(s) s.score = s.score + 1 end,
           draw = function(s) cls(0) end },
}, "title")
function _update() lib.update(); S:update() end
function _draw() S:draw() end
```

bmlib: text, bars and menus

Function	Description
`lib.input([over])`	the input standard for games: it puts `confirm`, `cancel`, `pause` and the actions of the game's scheme on the keymap and returns the table. `over.scheme` picks the scheme: `"scroller"` (the default: `act1` jump = B, `act2` attack = Y, `act3` = the A/B button that is not ok, `act4` = X, shoulders L1/R1), `"topdown"` (`act1` = B, `act2` = the other A/B button, `act3` = Y, `act4` = X) or `"stick"` (`act1` R2, `act2` L2, `act3` L1, `act4` R1, `act5` to `act8` = B, A, Y, X). `lib.input_scheme` tells which one runs. The other keys of `over` (`{ act3 = "x" }`) change single actions and are listed in `lib.input_exceptions`
`lib.textw(text, [scale])`	width and height in pixels with the font of `print` now (the longest line)
`lib.printc(text, y, [c, scale, x, w, grid])`	in the middle of the screen (or of `x..x + w`); `grid`: on the font's columns (text that stays still, as the menus). Returns the x
`lib.printr(text, x, y, [c, scale])`	ending at `x` (numbers aligned to the right)
`lib.prints(text, x, y, c, [shadow, scale])` / `lib.printo(...)`	with a shadow under it to the right / with an outline all round (black if the colour is missing)
`lib.bar(x, y, w, h, v, max, [c, back, border])`	a bar filled for `v` of `max` (life, stamina, loading)
`lib.blink([period, t])`	`true` half of the time, changing every `period` seconds (0.5): "press A"
`lib.timestr(secs, [tenths])`	`"m:ss"` (`"h:mm:ss"` from an hour), with tenths `"m:ss.d"`
`lib.btnr(i, [p, delay, rate])`	`btnp` that **repeats** while the button is held: the first frame, then after `delay` frames (15) every `rate` (4). `i` as for `btn()`. Call it in every frame
`lib.menu(items, [opt])`	a list to choose from: the items are texts or tables `{label=, value=, change=function(item, d), ok=function(item), off=true}`; `opt.p` the player, `opt.wrap` (`true`)
`M:update()`	up and down (repeating) move, skipping the `off` items, left and right call `change`, ok chooses (`ok(item)`; it returns the item and `"ok"`), back returns `nil, "back"`
`M:draw(x, y, [opt])`	the rows from `x, y`: `w` (width: the values on the right), `c`, `sel_c`, `bar`, `dim`, `scale`, `gap`

Function	Description
`lib.pause([opt])`	the **pause menu** every game had: Start (Esc) opens it when `opt.when()` is true (always if missing); RESUME, VOLUME (the console's: left and right), the rows of `opt.rows`, QUIT (`opt.quit()`, if given). `opt.color`, `opt.title`, `opt.scale`
`P:update()` / `P:draw()`	in `_update`: `if pause:update() then return end` (while it is open the game stands still); in `_draw`, after the game

```
local pause = lib.pause({ when = function() return S:is("play") end,
                          quit = function() S:go("title") end })
function _update()
  if pause:update() then return end
  lib.update(); S:update()
end
function _draw() S:draw(); pause:draw() end
```

bmlib: players on one console

`btn(i, p)`, `stick(p)` and `controller(p)` read player `p`; bmlib adds the screen where they join, the split screen and the colours. The SDK's *Versus 2D* template is a whole example.

Function	Description
`lib.PLAYER_COLORS`	the players' colours, those of the pads' lights: 1 blue, 2 red, 3 green, 4 pink (as `controller(p).color`)
`lib.pads()`	the numbers of the players who have a controller now, in order (`{1, 3}`)
`lib.party([opt])`	the screen where the players of one console join: each presses ok on their controller to be in, back to go out; Start of a player who is in begins, with at least `min` players (1); `max` (4); `join(p)`, `leave(p)`: functions called when one comes or goes. `P:update()` in `_update` returns the players in (their numbers, in the order they came) when the game begins, else `nil`; `P:draw([x, y, w, h])` a card for each place, in the player's colour, with their controller and the key to join; `P.list` who is in now
`lib.split(n, [opt])`	the screen cut into views `{x, y, w, h}` for `n` players (1-4): two side by side (`vertical` : one above the other), three or four in the corners (with three the bottom right stays free: a map, the scores); `gap` pixels between them (2); `x, y, w, h` the part of the screen (all of it if missing)

```
local party = lib.party({ min = 2 })
local views, cams

function _update()
  if not views then
    local who = party:update()              -- e.g. {1, 2}
    if who then
      views, cams = lib.split(#who), {}
      for i, v in ipairs(views) do cams[i] = lib.camera({ w = v.w, h = v.h }) end
    end
    return
  end
  -- ... cams[i]:follow(hero[i].x, hero[i].y)
end

function _draw()
  cls(0)
  if not views then party:draw(16, 80, SCREEN_W - 32, 200) return end
  for i, v in ipairs(views) do
    cams[i]:apply(v)
    cams[i]:map()
    draw_world()
  end
  clip()
  camera()
end
```

bmlib: sound, saves, animations, colours

Function	Description
`lib.jingle(notes, [voice, wave, vol])`	a short tune on the voice (3): `notes = { {note, secs, [wave, vol]}, ... }`, the note in Hz or by name (`"C5"`), 0 or `"-"` a rest; `lib.jingle(nil, voice)` stops it, `lib.jingling([voice])` says whether it still plays
`lib.store([k, [v]])`	a field of the cartridge's save (`save / saved`): `lib.store(k)` reads it, `lib.store(k, v)` writes it (on the SD card only if it changed: not in every frame), `lib.store()` the whole table
`lib.best(k, v)`	record `k`: `v` if it beats it (and saves it), and `true` if it is new
`lib.frame(frames, fps, [t])`	the element of `frames` that time `t` (`time()` if missing) shows at `fps` a second, looping
`lib.anim(frames, fps, [loop])`	an animation of its own: `a:update()` moves on by a frame and returns the element; `a.done` when one that does not loop (`false`) is over; `a:reset()`
`lib.mix(c1, c2, t) / lib.shade(c, k)`	between two `0xRRGGBB` colours / a colour `k` times brighter (0 black)

bmlib: the 3D builder

`lib.builder()` : a mesh made of convex pieces; each face of a piece is turned outwards by itself, so the order of the corners never matters.

Function	Description
`B:piece(points, triangles, [colour])`	a convex piece: `points = { {x, y, z}, ... }` , `triangles = { {i, j, k, [colour]}, ... }`
`B:box(x0, y0, z0, x1, y1, z1, colour, [top])`	a box between two corners; `top` : the colour of its top face
`B:tetra(p1, p2, p3, p4, colour)` / `B:quad(p1, p2, p3, p4, colour, nx, ny, nz)`	a tetrahedron / a flat quad seen from the side `nx, ny, nz`
`B:build()`	the mesh (`mesh()`); the methods return `B` , so they chain

```
local house = lib.builder()
  :box(-1, 0, -1, 1, 1.5, 1, 0xC0A080)
  :piece({ { -1.1, 1.5, -1.1 }, { 1.1, 1.5, -1.1 }, { 1.1, 1.5, 1.1 }, { -1.1, 1.5, 1.1 }, { 0, 2.4, 0 } },
         { { 1, 2, 5 }, { 2, 3, 5 }, { 3, 4, 5 }, { 4, 1, 5 }, { 1, 2, 3 }, { 1, 3, 4 } }, 0xA03020)
  :build()
```

Tests of the whole library: `make test-gameapi` (bmhost, `tests/gameapi/cart.lua` : every function with its cases, the buttons too with a script) and `test_game_api` in QEMU.

bmnet: opening the network and the lobby

The consoles find each other on the **LAN** (UDP broadcasts) or over the **internet** through a relay (`tools/overbit_relay.py` : a room passes every packet to the other consoles, as a broadcast would). One console **hosts** the match and the others **join**.

Function	Description
`net.open([opt])`	opens the network: `game` (4 characters, the game and its version: the packets of other games are not seen), `port` (47320, the same on every console), `relay` (`"name"` or `"name:port"` , 47310: over the internet; without, the LAN), `room` (4 characters, `"PLAY"` : the consoles of a room see each other on the relay), `name` (the player's name). `true` , or `false` and why (no network)
`net.close()`	leaves: the others know (in a match the seat's input becomes `false` ; when the host leaves the match ends for all), the socket closes. Call it from `_leave()` too
`net.host([opt])`	this console hosts a match: `max` players (2-8, 4), `info` (a line the others see in the list). It is seat 1
`net.hosts()`	the matches found in the last 3 seconds: `{id, name, players, max, info}` , a list, sorted by id
`net.join(id)`	asks console `id` for a seat: the event `"joined"` when it says yes, `"refused"` if it is full or has begun
`net.peers()`	the consoles in the room or the match: `{id, name, seat, me}` , by seat

[TO FILL: how to run the relay for a match over the internet; the sources only name `tools/overbit_relay.py --port N`]

bmnet: messages and events

Function	Description
`net.update()`	once in each `_update` : the packets that came, the lobby's announcements, the sure messages to send again. Returns the frame's **events**, a list of `{type = ...}` : `"join"` `( id, name, seat )`, `"leave"` `( seat, id )`, `"joined"` `( seat )`, `"refused"` , `"start"` `( seed, seat, seats, host, data )`, `"msg"` `( from, data, sure )`, `"lost"` `( why` : the host has gone), `"desync"` `( frame )`, `"error"` `( text )`
`net.send(data, [to])`	a message (a string up to 900 bytes) to everybody or to console `to` (an id): it may be lost or come after a newer one (positions)
`net.post(data, [to])`	a **sure** message: repeated until it arrives, and each console gets another's in the order they left (chat, a turn, a choice)

Use `net.send` for what is replaced by the next message (positions) and `net.post` for what must arrive (chat, a turn, a choice).

bmnet: the match in lockstep

For action games, every console runs the whole game with the same seed and only the players' inputs travel. A frame runs when everybody's inputs for it are there: the host gathers them and sends them to all.

Function	Description
`net.start([opt])`	(the host) starts the match with those in: `seed` (the same random numbers on every console; one by chance if missing), `delay` (frames between a press and its frame: 4 on the LAN, 7 through the relay), `data` (a string for everybody: the options chosen). Everybody gets the event `"start"` (the host too, at the next `net.update()`)
`net.input(v)`	(lockstep) my input for frame `net.frame + delay`: a 32-bit whole number (`net.pad(1)`). Once in each `_update` of the match, before `net.frames()`
`net.frames()`	the frames of the match that can run now: `for f, inputs in net.frames() do ... end`, `inputs[seat]` that seat's input (`false` once its player has left). One, two when the console is behind; none while an input is missing (`net.stall`: seconds since the last frame ran)
`net.check(hash)`	a number that sums up the game after the frame just run (positions, scores): the consoles compare them and a difference is the event `"desync"`. Every second is enough
`net.pad([p])` / `net.unpad(v)`	the 16 buttons of `pad(p)` and the left stick (8 bits each way) in 32 bits / back to buttons, `x`, `y` (-1..1)
`net.held(v, button)`	a button (`"a"`, `"left"` ... as `pad()`) held in an input of `net.pad`

BEWARE

The rules of lockstep. The match's game depends **only** on the frames' inputs and the seed: random numbers with `lib.rng(seed)` (never `math.random`, `time()` or `stat()`), nothing of the console (the camera, the quality) inside the simulation, no game state changed in `_draw`. The drawing may differ on every console.

- Two players on the same console in a network match put their inputs in the same number (16 bits each).
- `online(true)` is turned on by `bmnet` when the match begins: PS asks before leaving and calls `_leave()`.

bmnet: fields and a whole example

Fields

Field	Meaning
`net.state`	`"off"`, `"lobby"`, `"hosting"`, `"joining"`, `"joined"`, `"playing"` or `"lost"`
`net.me`	my id
`net.seat`	my seat
`net.seats`	the match's seats
`net.is_host`	`true` on the host
`net.frame`	the next frame
`net.seed`	the match's seed
`net.delay`	the match's delay in frames
`net.data`	the string given to `net.start`
`net.error`	the error text for the event `"error"` (unknown relay); also "the match is full or has begun", "the host left" and "the host is gone" (these come with the events `"refused"` and `"lost"`). `net.open` resets it to `nil`
`net.stall`	seconds since the last frame ran, while `net.frames()` waits (reset in `net.frames()` and at the start of a match; it grows 1/60 per `net.update()`)

A whole example

```
local net = require "bmnet"
local lib = require "bmlib"
local rng

function _init() net.open({ game = "MYG1" }) end
function _leave() net.close() end

function _update()
  for _, e in ipairs(net.update()) do
    if e.type == "start" then rng = lib.rng(e.seed) end
  end
  if net.state == "lobby" and btnp("a") then net.host({ max = 2 }) end
  if net.state == "hosting" and btnp("start") then net.start() end
  if net.state == "playing" then
    net.input(net.pad(1))
    for _, inputs in net.frames() do step(inputs) end   -- the game, the same everywhere
  end
end
```

Trying it on the PC

Two `bmhost` on the same PC are two consoles: `BMHOST_NET_ID=0` and `BMHOST_NET_ID=1`, with `--realtime`; the relay runs with `tools/overbit_relay.py --port N`.

`make test-bmnet` does it: the lobby, sure messages with a fifth of the packets lost (`loss = 0.2` in `net.open`, for tests only), a lockstep match the same on both consoles, a player leaving; on the LAN and through the relay.

Open points

- **Tutorial: a platformer step with bmlib**: how to paint the tiles and set their flags in the editors; this volume only describes the library
- **Tutorial: a platformer step with bmlib**: picture to take: shots/api-lib-platformer.png
- **Tutorial: a game for two over the network with bmnet**: picture to take: shots/api-lib-lobby.png
- **bmnet: opening the network and the lobby**: how to run the relay for a match over the internet; the sources only name `tools/overbit_relay.py --port N`